

Sleep-Time Compute: 지속형 AI 기억을 위한 조건부 Scaling Axis

Samsung SAIT Neural Memory Study

Source freeze: 2026-08-05

Abstract

Persistent AI 는 release-time training 과 current-query inference 사이에서 accumulated wake experience 를 later-wake reusable state 로 전환해야 한다. 본 논문은 이 deferred lifecycle 을 sleep-time compute(STC) 로 엄격히 정의하고, 73 개 primary/official source 의 date-frozen audit 를 strongest alternatives 와 비교한다. 공개 증거는 external background synthesis 에는 production maturity 가 있지만 broad per-user foundation-weight sleep 에는 없음을 보인다. 이에 따라 auditable external system-of-record 에서 stable high-reuse subset 만 reversible parametric state 로 승격하는 hybrid 를 조건부 base case 로 제시한다. Universal scaling law 는 주장하지 않고 complete-state identities, reuse/cadence break-even, capacity knee, state-migration crossover 와이를 반증할 repeated-cycle benchmark 를 도출한다. 결과는 STC 라는 명칭의 승리보다 multi-clock, multi-medium memory lifecycle 이 주류가 될 가능성을 지지한다.

Artifact set: 70-page Full Study · separate Conference Appendix · 57-page Training Background

1 문제와 operational definition

Pre/post-training 은 shared capability 를 배포 전에 만든다. Test-time inference 와 learning^[배경] 은 현재 query 또는 wake episode 의 latency boundary 안에서 reasoning, retrieval, fast-state update 를 수행한다. 그 사이에 persistent agent 가 겪은 correction, tool outcome, procedure 와 preference 를 나중 wake 에 재사용 가능한 상태로 바꾸는 phase 가 비어 있다.

본 논문은 STC 를 다음 다섯 조건의 교집합으로 정의한다: (1) input 은 실제 wake lifetime 에서 축적되고, (2) 실행은 response-critical boundary 뒤로 deferred 되며, (3) 결과는 다음 wake 에도 남는 reusable state 이고, (4) candidate 는 validation 과 publication boundary 를 거치며, (5) cost 와 benefit 은 later-wake trace 에서 측정한다. Ordinary offline batching, within-query chain-of-thought, passive cache eviction, source lineage 없는 background summary, wake data 와 무관한 generic retraining 은 제외한다.

Timing 과 medium 은 직교한다. Sleep 은 text/vector/graph external state, latent/recurrent state, LoRA adapter^[배경], expert 또는 shared weight 를 바꿀 수 있다. 반대로 같은 medium 도 wake update 와 sleep update 를 모두 가질 수 있다. 이 분리 없이 “external 인가 parametric 인가” 만 물으면 scheduling isolation, validation 과 reuse amortization 의 효과를 medium 자체의 장점으로 오인한다.

핵심 novelty 는 새 optimizer 하나가 아니다. Admission, operator, destination, cadence, validation 과 retirement 를 하나의 lifecycle optimization 으로 묶고, quality gain 을 complete state 와 later-wake reuse 에 대해 평가하는 framing 이다. 이 framing 은 external-only system 에도 적용되므로 parametric result 가 실패해도 연구 가치가 남는다.

Wake-test-sleep lifecycle

경계는 clock time이 아니라 response-critical dependency다

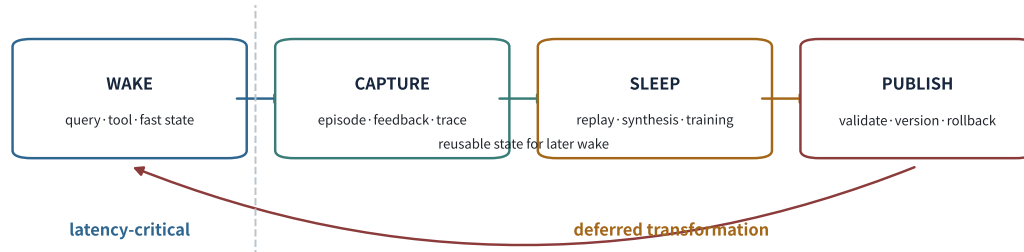


Figure 1: Wake-test-sleep lifecycle 과 reusable-state boundary. Sleep 은 candidate 를 만들 뿐 validation 전 live state 를 직접 바꾸지 않는다.

출처/제작: Original synthesis; SRC-STC-0014, SRC-STC-0021, SRC-STC-0052 대체 설명: Wake event 가 snapshot, sleep transform, validation 과 atomic publish 를 거쳐 다음 wake 에서 사용되는 순환 구조.

Table 1: 본 연구의 기여와 의도적으로 남긴 non-claim

기여	산출물	Non-claim
operational definition	causal boundary 와 timing×medium taxonomy	biological stage 의 one-to-one 모사
evidence synthesis	73-source date-frozen claim graph	publication count 기반 trend score
matched decision	problem 별 strongest baseline 과 hybrid gate	fixed model straw-man 대비 우위
scaling agenda	identity, break-even, knee, fit protocol	universal sleep-FLOP law
systems bridge	transactional publish 와 migration accounting	세 physical cluster 가 항상 필요
benchmark	repeated-cycle matched lifecycle design	현재 production dominance 입증

2 Evidence 와 strongest alternatives

2026-08-05 에 동결한 73-source corpus 는 biological/computational replay, continual learning [배경], external agent memory, neural/test-time memory, product background synthesis, capacity/security 와 negative evidence 를 포함한다. OpenAI Dreaming 과 Mem0 Dream 은 external memory background synthesis 를 공식 설명하고, Letta 는 post-response asynchronous implementation 을, Zep/Graphiti 는 temporal episode provenance 를 공개한다. (OpenAI 2026; Letta 2026b; Mem0 2026; Zep 2026) 반면 explicit parametric sleep 은 “Language Models Need Sleep” 의 research-scale evidence 가 중심이다. (Behrouz et al. 2026)

Evidence maturity 와 expected value 를 평균하지 않았다. Product release(E4) 는 lifecycle 의 존재를 지지하지만 comparative optimum 을 지지하지 않는다. Broad per-user weight update 는 retention, delete, rollback, security 와 fleet economics 를 함께 보여 주는 공개 trace 가 없다. Sequential fact learning 은 information 이 weight 에 남아도 normal behavior 에서 접근 불가능해질 수 있음을 보이고, memory poisoning 은 external state 도 자동으로 안전하지 않음을 보인다. (O'Neill 2026; Chen et al. 2024)

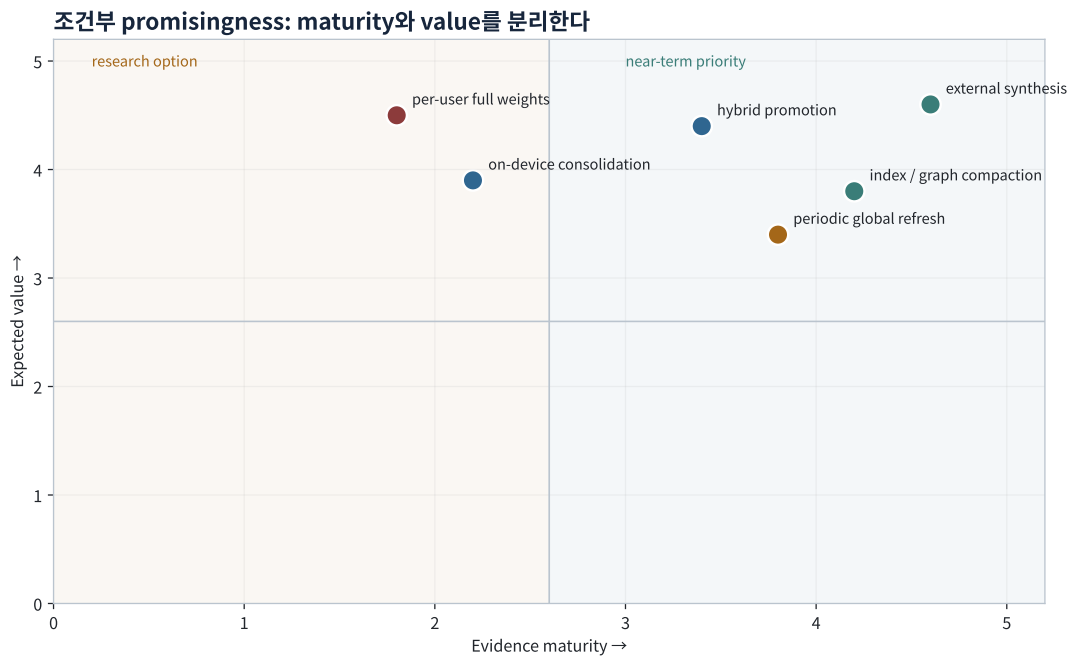


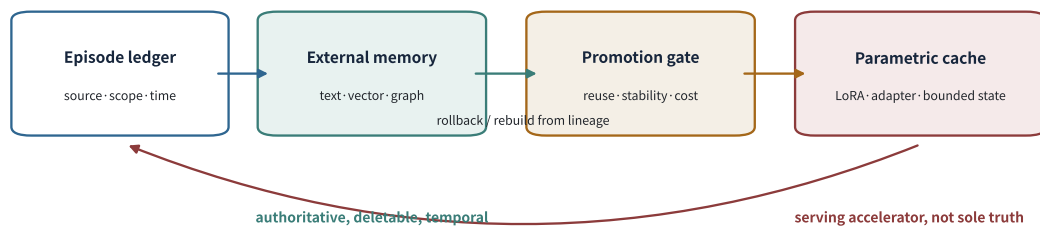
Figure 2: 현재의 conditional promisingness. External synthesis 는 높은 maturity, selective parametric promotion 은 높은 upside 와 낮은 end-to-end maturity 를 갖는다.

출처/제작: Original synthesis; SRC-STC-0052, SRC-STC-0063, SRC-STC-0021 대체 설명: External, learned writer, adapter promotion, broad weight update 를 maturity 와 expected-value 축에 놓은 산점도.

One-off query 에서는 STC 가 일반적으로 최선이 아니다. High-churn fact 는 exact evidence, temporal validity 와 deletion 이 쉬운 external destination 이 유리하다. Stable skill 과 procedure 는 repeated retrieval/context cost 를 parametric compile 로 선불 지불할 수 있다. 따라서 가장 방어적인 구조는 external source-of-record 를 유지하고, high-reuse stable subset 만 versioned adapter 또는 bounded state 로 승격하는 hybrid 다.

Table 2: Problem 별 strongest non-STC baseline 과 STC 가 이길 조건

Problem	Strongest baseline	STC 가 후보가 되는 조건
one-off long document	long context/RAG	reuse 와 query predictability 가 높음
volatile exact fact	temporal graph/RAG	external conflict resolution; weight write 는 열세
repeated procedure	strategy memory/cache	held-out transfer 가 compile cost 를 상쇄
rare factual patch	ROME/MEMIT/source edit	multi-edit integration 과 validation 이 필요
global knowledge	periodic continued PT	local/private change 가 release cadence 보다 빠름
continual skill	replay+PEFT/global refresh	bounded state 에서 old/new/plasticity 유지
serving efficiency	prefix/state cache	reusable state compatibility 와 hit rate 가 충분

Hybrid: external system-of-record → selective reversible promotion**Figure 3:** External system-of-record 와 selective reversible promotion. Parametric artifact 는 authoritative memory 가 아니라 rebuild 가능한 serving cache 다.

출처/제작: Original synthesis; SRC-STC-0030, SRC-STC-0066 대체 설명: Canonical episode 에서 external views 와 promotion gate 를 지나 adapter cache 로 가고 lineage 를 따라 rollback 하는 구조.

3 Conditional mainstream verdict

현재 base case 는 near-term external dominant, high-reuse domain 에서 hybrid promotion 으로
의 진화, global common knowledge 에 대한 periodic refresh 의 공존이다. STC broad adoption
은 hundreds-cycle retention, valid-reuse prediction, end-to-end delete/rollback, stable multi-
resource queue, state-migration economics 가 필요하다. 명칭은 dreaming, reflection, synthesis,
self-evolution 또는 memory 로 남을 수 있으므로 adoption 은 keyword 가 아니라 operational
lifecycle 로 추적한다.(OpenAI 2026; Google Research 2026; Letta 2026a; Mem0 2026)

Mainstream confidence 를 올릴 signal 은 public per-user artifact lifecycle, adapter/index 의 base-
version churn cost, 100–1,000 cycle retention–plasticity curve, reuse predictor calibration, derived-
state delete fault injection, background job bytes/energy/queue trace 다. 반대로 validation 과
state fan-out 이 saved inference 보다 빠르게 커지거나, high reuse 에서도 external baseline 과
crossover 가 없으면 parametric scope 를 줄여야 한다.

Table 3: 상호 배타적 예측이 아닌 workload-conditioned mainstream scenarios

Scenario	성립 조건	가장 강한 falsifier
external dominant	provenance/delete 우선, frequent base churn	stable procedure 에서 persistent quality loss
hybrid promotion	reuse prediction, reversible adapter, lineage	dual-state/version cost 가 saving 을 상쇄
periodic refresh	global amortization, local retention 제한	release 사이 local procedure 변화가 큼
STC niche	high reuse 가 일부 domain 에 만 존재	general assistant background synthesis 확산
STC broad	hundreds-cycle 안정성, queue/delete/migration 해결	validation·fan-out·privacy cost 의 초과 성장

시나리오들은 같은 조직 안에서도 공존할 수 있다. Consumer factual memory 는 external dominant,
enterprise procedure 는 cohort adapter hybrid, universal knowledge 는 periodic refresh 일
수 있다. 따라서 “STC 가 mainstream 인가” 의 단위는 전체 모델이 아니라 workload×content
class×destination 이다.

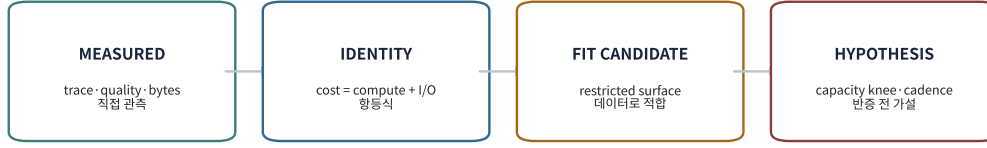
4 Scaling: identity, break-even, fit candidate

Universal STC scaling law 는 아직 없다.(Morris et al. 2025; Colaco and Lahjouji 2026; Unknown
organization 2026) Complete state 를

$$B_{retained} = B_r + B_e + B_l + B_p + B_a + B_v \quad (1)$$

로 기록한다. Raw evidence, external views, latent/KV, parametric state, auxiliary optimiz-
er/router 와 version/recovery 를 모두 포함한다. Peak physical state 는 candidate, pinned pre-
decessor, migration, workspace, replica, base-model residence 와 execution memory 를 같은
timestamp 에서 더한다. Full replay 는 $m_i = m$ 일 때 lifetime examples 가 $mT(T+1)/2 = \Theta(T^2)$
이므로 constant admission fraction 만 줄여서는 asymptotic burden 이 바뀌지 않는다.

수식의 지위를 섞지 않는다



universal 'more sleep FLOPs → better memory' law: not established

Figure 4: Scaling statement 의 네 등급. 보편 법칙으로 갈수록 held-out prediction 과 cross-condition evidence 가추가로 필요하다.

출처/제작: Original synthesis; SRC-STC-0035, SRC-STC-0050 대체 설명: Accounting identity, analytical break-even, fit candidate, untested hypothesis 의 네 단계.

Quality 가 같고 use 가 stationary 라는 제한에서 compile reuse break-even 은

$$n^* = \frac{C_{compile} + C_{maint}}{\Delta C_{use}}. \quad (2)$$

Validity 와 discount 가 있으면 $R_i(H) = \int_0^H \lambda_i(t) S_i(t) e^{-\omega t} dt$ 를 사용한다. Promotion 은 $R_i(H) \Delta U_{ij}$ 가 compile, serve, governance, risk 의 합보다 클 때만 후보가 된다. Raw memories 또는 sleep FLOPs 보다 valid reusable evidence per complete lifecycle cost 가 중요한 이유다.

Cadence 도 monotone 하지 않다. Run cost C_s 와 staleness $a\tau^p$ 의 제한된 모델은

$$\bar{C}(\tau) = \frac{C_s}{\tau} + a\tau^p, \quad \tau^* = \left(\frac{C_s}{ap} \right)^{1/(p+1)} \quad (3)$$

를 준다. Burst, hard deadline, affinity 와 capacity change 에서는 queue trace 로 다시 측정한다. Fixed finite state 에는 physical occupancy 전에 behavioral reachability 또는 plasticity 가 무너지는 capacity knee 가 있을수 있다. 이를 global power law 로 숨기지 않고 segmented/nonparametric competitor 와 비교한다.

Cadence 식의 실험적 의미는 per-event, fixed interval, queue pressure, capacity trigger, learned policy 를같은 admitted evidence 와 destination 에서 비교하는 것이다. Staleness 뿐 아니라 sleep job fixed cost, batch efficiency, old-task interference, wake P99 를 함께 기록해야 한다. Average utilization 이 1 보다 낮아도 state affinity 와 validator 병목 때문에 P99 age 가 발산할 수 있다.

첫 fit candidate 는

$$\Delta U = A \left[1 - e^{-k F_s^\alpha E_{eff}^\beta R_v^\gamma} \right] - \Phi_{int} - \Phi_{stale} - \Phi_{risk} \quad (4)$$

처럼 evidence quality 와 reuse 를 분리하고 interference, staleness, risk 를 뺀다. 이는 law 가 아니다. 둘 이상의 model, workload, medium, hardware 에서 strongest baseline 과 total lifecycle cost 를 맞추고 held-out condition 을예측하며 recursive depth 와 repeated-cycle control 을 통과해야 제한된 empirical law 로 승격할 수 있다.

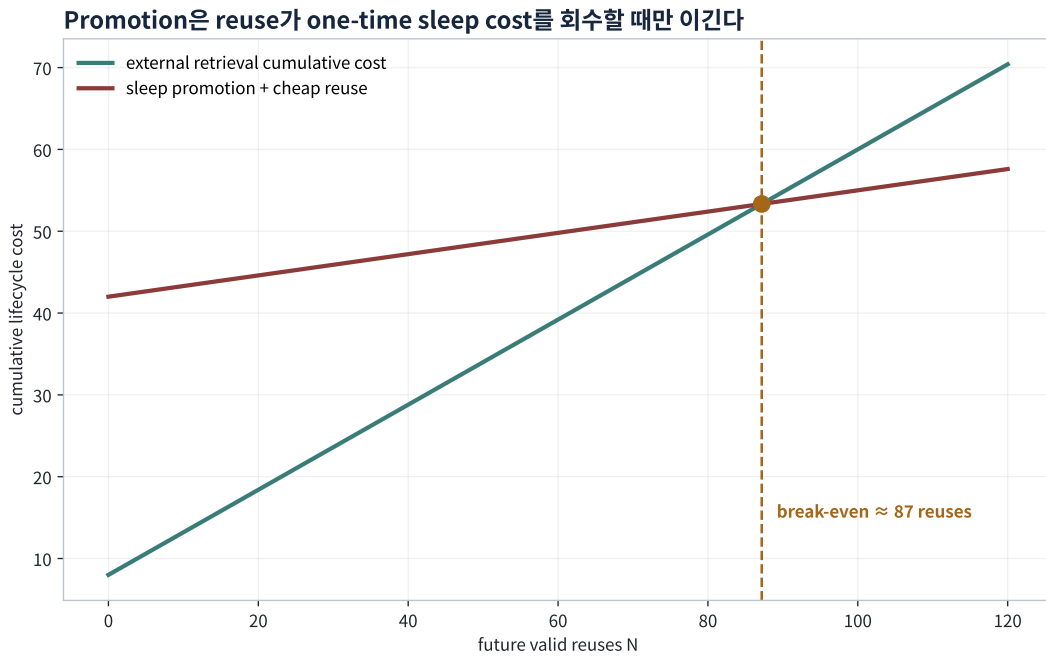


Figure 5: Future valid reuse 가 one-time compile cost 를 상쇄하는 analytical crossover.

출처/제작: Original analytical redraw; SRC-STC-0027, SRC-STC-0052 대체 설명: Compile 비용과 reuse 누적 saving 의 교차점.

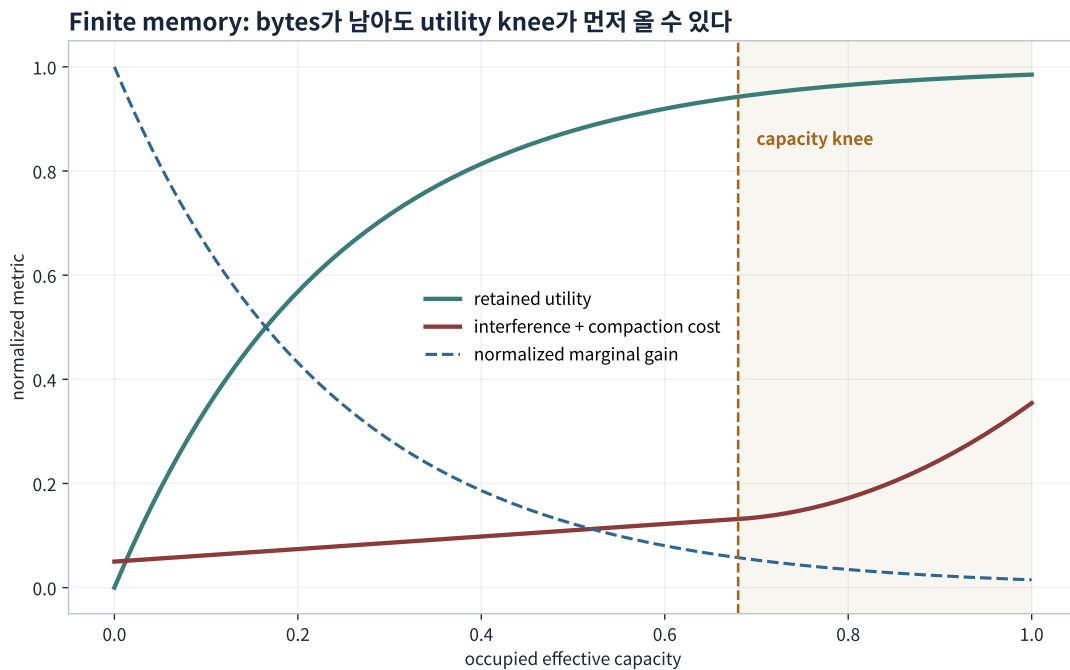


Figure 6: Finite-state capacity knee hypothesis. Behavioral reachability 와 plasticity 는 physical occupancy 보다 먼저 임계값을 넘을 수 있다. 곡선은 측정 결과가 아니라 preregistration target 이다.

출처/제작: Original hypothesis redraw; SRC-STC-0034, SRC-STC-0036 대체 설명: Evidence load 에 따라 retention 과 plasticity 가 서로 다른 knee 에서 감소하는 곡선.

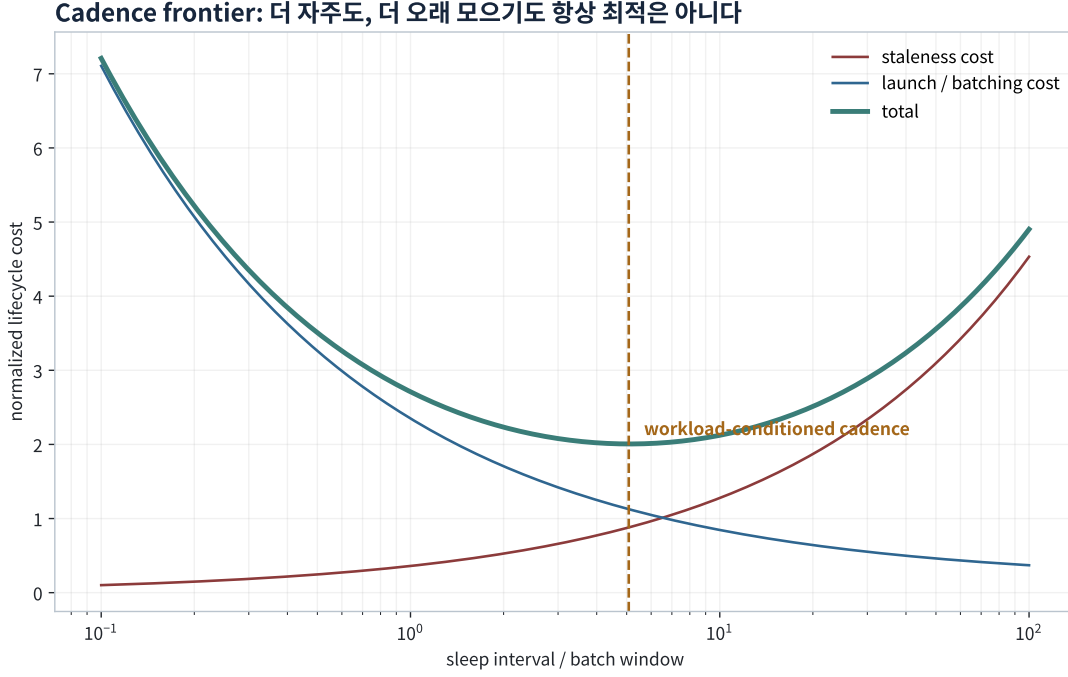


Figure 7: Cadence frontier. 짧은 interval 의 overhead 와 긴 interval 의 staleness 가 내부 optimum 을 만든다.

출처/제작: Original analytical redraw; SRC-STC-0020, SRC-STC-0063 대체 설명: Batching cost 와 staleness cost 의 합이 U 자 곡선을 만드는 설계 공간.

5 Wake-sleep infrastructure 와 device implication

Logical architecture 는 wake plane, sleep candidate plane, memory/control plane 으로 나뉜다. Sleep job 은 immutable snapshot 과 published manifest 를 읽고 candidate 만 만든다. Validation 뒤 complete serving head 를 compare-and-swap 하며, delete/authorization epoch 가 중간에 바뀌면 publish 가 실패한다. External raw evidence 는 허용 기간 동안 source-of-record 이고, summary, index, graph, latent state 와 adapter 는 lineage 를 가진 materialized view 다.

Separate sleep cluster 가 항상 효율적인 것은 아니다. Link 별 이동 cost 를

$$C_{move} = \sum_{\ell} (B_{\ell} p_{\ell} + T_{\ell} v_{delay}) \quad (5)$$

로 두고 source 뿐 아니라 base model, optimizer, candidate, validation, predecessor 를 센다. Shared priority, time-partitioned, separate, shard-local placement 를 P99 wake interference, sleep job age, physical peak 와 total movement 로 비교한다.

Memory-device 관점의 강건한 workload 는 versioned derived state, temporal index rewrite, snapshot/COW, heterogeneous KV/recurrent/adapter state cache, provenance lookup, secure deny/rebuild 와 lifecycle telemetry 다. 이 traffic 은 external-dominant 미래에도 남는다. 다만 구체적 제품 hypothesis 는 patent-first hold 이며 이공개 원고는 bandwidth, endurance, tail latency, bytes moved 와 rollback time 의 benchmark question 만 제시한다.

장치 평가는 peak GB/s 하나로 끝나지 않는다. $B_{peak, physical}$, device-level write amplification, sequential episode scan 과 random graph/index access, manifest/lineage P99, adapter page-in, secure erase, rollback RTO 를 workload phase 별로 기록한다. Near-data transform 은 saved link bytes 에서 local execution 과 validation cost 를 뺀 end-to-end 결과로 판단한다.

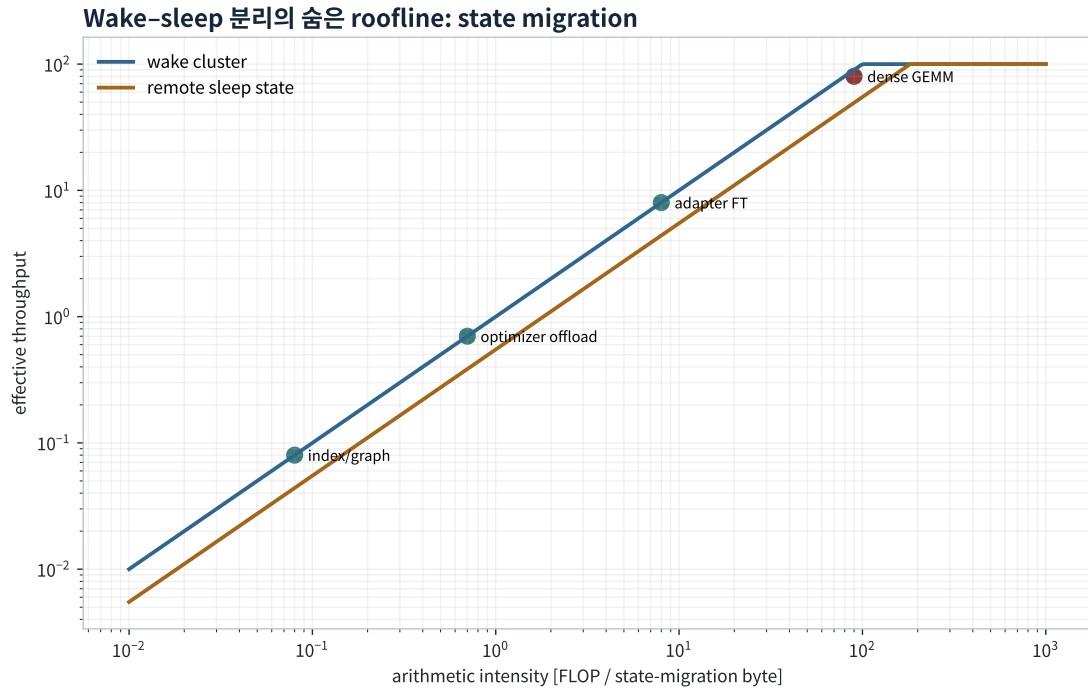


Figure 8: State-migration roofline. Source-to-delta ratio, model residency 와 link bandwidth 가 placement crossover 를 정한다.

출처/제작: Original systems synthesis; SRC-STC-0022, SRC-STC-0059 대체 설명: Shared, separate, shard-local placement 의 이동량과 compute efficiency trade-off.

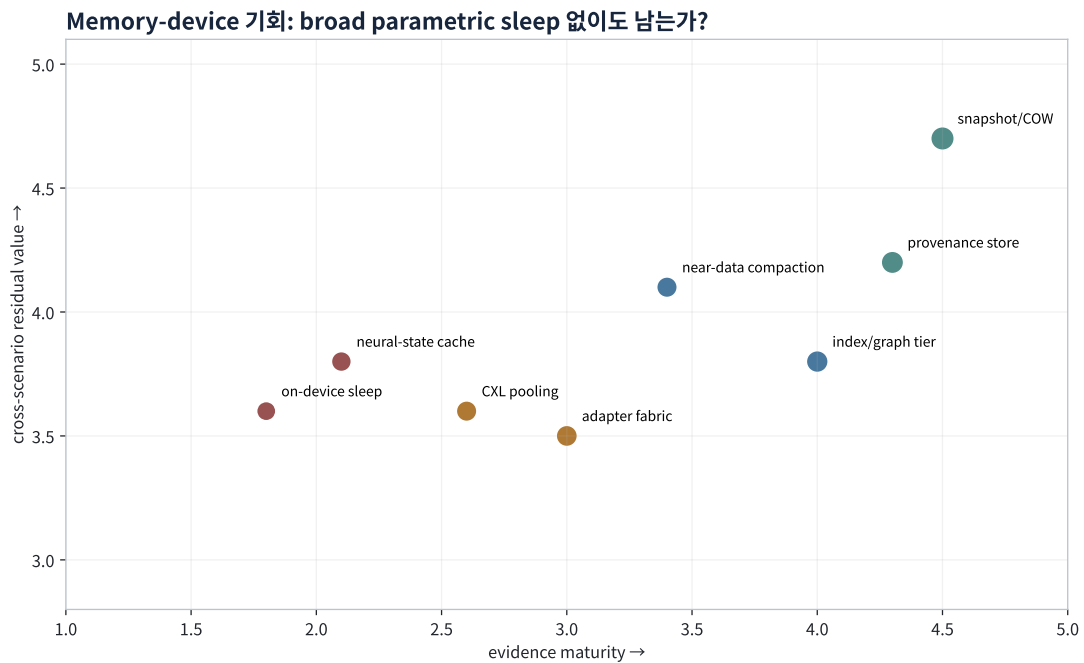


Figure 9: Device research map. Evidence maturity 와 STC adoption 이 약해져도 남는 residual workload value 를 분리한다. 좌표는 market forecast 가 아니다.

출처/제작: Original research map; SRC-STC-0056, SRC-STC-0065 대체 설명: Snapshot, provenance, mutable index, state cache, pooling, local compaction 후보를 두 축에 배치.

6 Falsifiable benchmark 와 limitations

한 cycle 은 wake capture, pre-sleep probe, immutable snapshot, candidate, validate/publish, post-sleep 와 delayed next-wake probe 다. Default 128 cycles 와 $R \in \{0, 1, 4, 16\}$, active-capacity $\phi \in \{1, 1/4, 1/16\}$ 을 교차한다. Raw external, compressed external, latent/KV, user LoRA, static mixture, past-only hybrid router 를 동일 event/query/resource 에서 비교하며 future reuse 와 correction 은 oracle ledger 에격리한다.

반증 가능한 research → system → device agenda

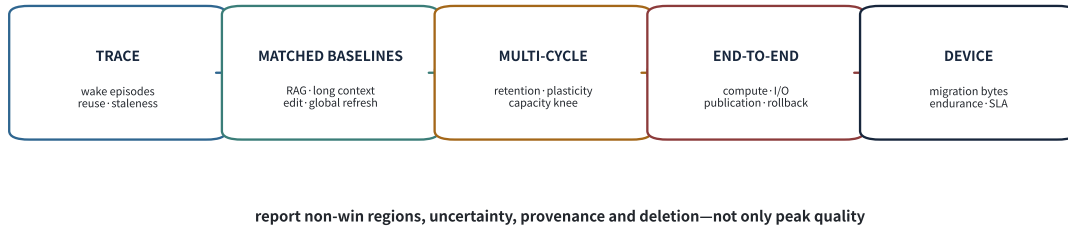


Figure 10: Matched baseline 에서 multi-cycle, governance, systems 와 device trace 로 이어지는 반증 가능한 benchmark agenda.

출처/제작: Original synthesis; SRC-STC-0036, SRC-STC-0050, SRC-STC-0056 대체 설명: Trace, matched methods, repeated cycle, end-to-end control, device measurement 의 순서.

Metric 은 lifetime utility AUC, acquisition/retention/plasticity, exact/temporal/procedural/rare-tail, false memory, poison, access deny/physical removal/behavioral residual, TTFT/ITL, queue age, retained/peak bytes, migration, write amplification, energy 와 base compatibility 를 포함한다. Timeout/OOM/deadline miss 는 missing 이 아니라 RESOURCE_FAILURE 로 charge 한다.

본 연구는 biological stage 대응, universal monotone law, parametric dominance, product comparative optimality 를 주장하지 않는다. DANN/SSRN record 는 full method 와 result 가 없어 load-bearing evidence 에서제외했다.(Tutuncuoglu 2025) Search 는 date-bounded snapshot 이며 5 active frontier 와 5 measurement gap 을남긴다. 100–1,000 cycle, independent product evaluation, delete fault injection 과 measured hardware trace 가후속 confidence 를 결정해야 한다.

구체적 claim-down rule 은 결과를 보기 전에 고정한다. Reuse interaction 이 없으면 promotion law 를, behavioral knee 가 없으면 capacity hypothesis 를, shared fleet 가 모든 realistic trace 에서 더 낮다면 dedicated sleep cluster 를 기각한다. Archive 를 보존한 active-memory 감소는 total compression 이 아니며, tombstone 은 physical deletion 이나 unlearning 이 아니다. Simulation 은 특정 accelerator 나 memory device 의 absolute latency/energy 를 지지하지 못한다.

7 결론

STC 는 persistent AI 의 실제 lifecycle 축이며 external background consolidation 에서 이미 product momentum 을가진다. 그러나 continuous per-user foundation-weight update 는 현재 strongest default 가 아니다. 가장유망한 다음 단계는 auditable external evidence 위의 selective reversible

Failure mode마다 lifecycle control을 붙인다

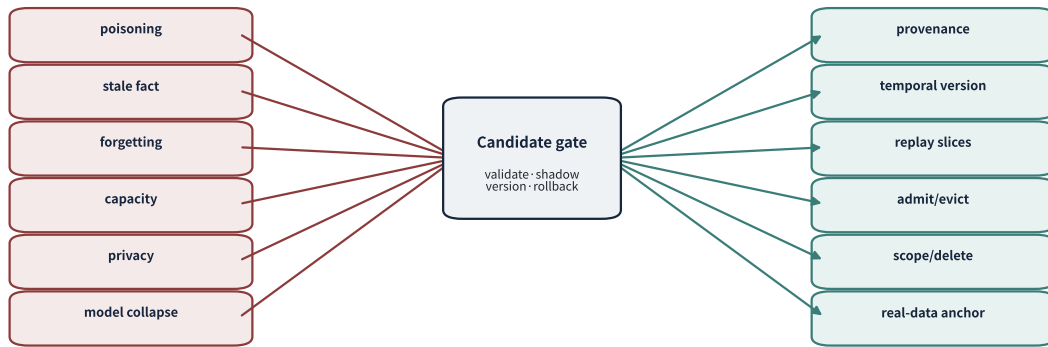


Figure 11: Lifecycle failure 와 control. Accuracy validation 하나만으로 poison, stale publish, rollback resurrection 과 derived-state deletion 을 막지 못한다.

출처/제작: Original synthesis; SRC-STC-0042, SRC-STC-0053 대체 설명: Admission 부터 delete 까지 failure 와 provenance, quarantine, CAS, deny/rebuild control 의 대응.

promotion 이고, 성공 metric 은 accuracy versus sleep FLOPs 가 아니라 valid later-wake utility per complete lifecycle resource 다. 연구의 우선순위는 더 큰 sleep model 이 아니라 admission, destination, cadence, validation, retirement policy 와 state movement 를 함께 측정하는 것이다.

AI assistance disclosure. During the preparation of this work the authors used Codex GPT-5.6 for evidence synthesis, LaTeX drafting, and artifact QA. All content was reviewed and edited by the authors, who take full responsibility for the accuracy and integrity of the work.

참고문헌

- Behrouz, Ali, Farnoosh Hashemi, Adel Javanmard, and Vahab Mirrokni (2026). *Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories*. Frozen source SRC-STC-0021; type=paper; accessed=2026-08-05. arXiv: 2606.03979. URL: <https://arxiv.org/abs/2606.03979v2> (visited on 08/05/2026).
- Chen, Zhaorun, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li (2024). *AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases*. Frozen source SRC-STC-0042; type=paper; accessed=2026-08-05. arXiv: 2407.12784. URL: <https://arxiv.org/abs/2407.12784v1> (visited on 08/05/2026).
- Colaco, Ashwin Gerard and Nada Lahjouji (2026). *What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents*. Frozen source SRC-STC-0035; type=paper; accessed=2026-08-05. arXiv: 2607.08032. URL: <https://arxiv.org/abs/2607.08032v1> (visited on 08/05/2026).
- Google Research (2026). *ReasoningBank: Enabling Agents to Learn from Experience*. Version web-capture:2026-08-05. Frozen source SRC-STC-0054; type=official release note; accessed=2026-08-05. URL: <https://www.research.google/blog/reasoningbank-enabling-agents-to-learn-from-experience/> (visited on 08/05/2026).

- Letta (2026a). *Letta Code Reflection Launcher*. Frozen source SRC-STC-0060; type=official repository; accessed=2026-08-05. URL: <https://github.com/letta-ai/letta-code/blob/v0.28.18/src/cli/helpers/reflection-launcher.ts> (visited on 08/05/2026).
- (2026b). *Letta SleepTime Multi-Agent V4*. Frozen source SRC-STC-0059; type=official repository; accessed=2026-08-05. URL: https://github.com/letta-ai/letta/blob/0.16.8/letta/groups/sleeptime_multi_agent_v4.py (visited on 08/05/2026).
- Mem0 (2026). *Mem0 Dream*. Version web-capture:2026-08-05. Frozen source SRC-STC-0063; type=official documentation; accessed=2026-08-05. URL: <https://docs.mem0.ai/platform/features/dream> (visited on 08/05/2026).
- Morris, John X., Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar (2025). *How much do language models memorize?* Frozen source SRC-STC-0034; type=paper; accessed=2026-08-05. arXiv: 2505.24832. URL: <https://arxiv.org/abs/2505.24832v3> (visited on 08/05/2026).
- O'Neill, Charles (2026). *Can a Language Model Learn Facts Continually in Its Weights?* Frozen source SRC-STC-0036; type=paper; accessed=2026-08-05. arXiv: 2607.11020. URL: <https://arxiv.org/abs/2607.11020v2> (visited on 08/05/2026).
- OpenAI (2026). *Dreaming: Better Memory for a More Helpful ChatGPT*. Version web-capture:2026-08-05. Frozen source SRC-STC-0052; type=official release note; accessed=2026-08-05. URL: <https://openai.com/index/chatgpt-memory-dreaming/> (visited on 08/05/2026).
- Tutuncuoglu, Bekir Tolga (2025). *Dream-Augmented Neural Networks: Harnessing Synthetic Sleep for Continual Learning and Zero Forgetting*. Frozen source SRC-STC-0039; type=paper; accessed=2026-08-05. DOI: 10.2139/ssrn.5402490. URL: <https://doi.org/10.2139/ssrn.5402490> (visited on 08/05/2026).
- Unknown organization (2026). *Memory-Statistics Tradeoff in Continual Learning with Structural Regularization*. Version web-capture:2026-08-05. Frozen source SRC-STC-0050; type=paper; accessed=2026-08-05. URL: <https://openreview.net/forum?id=qfEqXJn1B4> (visited on 08/05/2026).
- Zep (2026). *Graphiti: Temporal Knowledge Graphs for Agent Memory*. Frozen source SRC-STC-0065; type=official repository; accessed=2026-08-05. URL: <https://github.com/getzep/graphiti/blob/aab852df94413fd0d55cbea2b7886173020281d5/README.md> (visited on 08/05/2026).